3

Build System Design

Data Flow Diagrams • Physical Data Models • Documentation

SWDBS401 Backend System Design — RQF Level 4

Learning Outcome 3

Total: 32 Hours

Table of Contents

3.1

Development of Data Flow Diagram

12 Hours

3.2

Application of Physical Data Model

8 Hours

3.3

Documentation of System Design

12 Hours

3.1

Development of Data Flow Diagram

Understanding, designing, and applying DFDs in system analysis

What is a Data Flow Diagram?

A Data Flow Diagram (DFD) is a traditional visual representation of how information flows within a system.

- A neat and clear DFD can depict the right amount of the system requirement graphically
- It can be manual, automated, or a combination of both
- Used by software engineers, IT, ecommerce, and product management professionals
- Helps stakeholders understand system requirements at a glance

Elements of DFD

1 Processes

Circles or ovals that transform inputs into outputs. Each process performs a specific action on data passing through the system.

2 Data Flows

Arrows showing the direction of data movement between processes, data stores, and external entities.

3 Data Stores

Open-ended rectangles that represent repositories for data. Data is stored here for later use by processes.

4 External Entities

Squares or rectangles representing sources or destinations outside the system boundary.

DFD Symbols



Process



Data Flow



Data Store



External Entity

Symbol Descriptions

Process (Circle/Oval)

Transforms input data into output data. Represents activities or actions performed on data.

Data Flow (Arrow)

Shows the path and direction data moves between processes, stores, and external entities.

Data Store (Open Rectangle)

Represents a repository where data is held for later retrieval. Like a database or file.

External Entity (Square)

Represents entities outside the system that interact with it, such as users or other systems.

Rules of Drawing a DFD

01

Use Consistent Symbols

Stick to standard DFD symbols throughout the entire diagram for clarity and professionalism.

02

One Input, One Output

Each process needs at least one input and one output data flow to be valid.

03

Label Everything

Clearly label all processes, data stores, and data flows with descriptive names.

04

No Control Flows

DFDs focus on data movement, not control decisions or sequential logic.

05

Maintain Hierarchy

Higher-level diagrams show overview; lower-level diagrams show increasing detail.

Types of Data Flow Diagram

L0

Level 0

Context Diagram

- Highest-level DFD showing the entire system as a single process
- Overview of all external entities interacting with the system
- No internal details visible
- Defines the system boundary

L1

Level 1

Detailed Processes

- Breaks major processes into sub-processes
- Shows data stores within the system
- Reveals internal data flows
- More detailed than Level 0

L2

Level 2

Function Decomposition

- Further decomposes sub-processes from Level 1
- Plans or records specific details about system functioning
- Each sub-process depicted separately
- Maximum detail level

Level 0 DFD — Context Diagram

LEVEL 0

Also known as a **Context Diagram** — the highest-level view of a system

- Designed as an abstraction view showing only the system boundary

Shows the entire system as a **single process** (bubble/circle)

- Input and output data indicated by arrows connecting external entities
- Shows relationship to external entities outside the system boundary

No internal details shown — only external interactions are visible

> **Key Concept:** Level 0 provides a "bird's eye view" of the system, focusing on what data enters and leaves, not how it is processed internally.

Level 1 DFD — Detailed Processes

LEVEL 1

The context diagram (Level 0) is **decomposed into multiple processes**

Highlights the **main functions** of the system

- Breaks the high-level single process of Level 0 into sub-processes

Shows **data flows** and **data stores** for each sub-process

Reveals internal workings of the system at a **moderate level of detail**

> **Key Concept:** Level 1 shows what happens inside the system — the major sub-processes, how data flows between them, and where data is stored.

Level 2 DFD — Function Decomposition

LEVEL 2

Goes **one step deeper** into parts of Level 1

Further **decomposes sub-processes** from Level 1 into more granular steps

Plans or records **specific detail** about system functioning

Each sub-process is depicted as a **separate process**

- Data flows and data stores associated with each are clearly shown

> **Key Concept:** Level 2 provides maximum detail. You can continue to Level 3, 4, etc., but most systems are sufficiently documented at Level 2.

Software Tools for Designing DFDs

V

Microsoft Office

Visio — dedicated diagramming tool with DFD templates

PowerPoint — simple DFDs using shapes and connectors

- Widely available and easy to learn
- Good for basic diagramming needs

VP

Visual Paradigm

- Dedicated UML and DFD modeling tool
- Robust DFD features with multi-level support
- Supports UML diagrams and other notations
- Templates and team collaboration features

ED

E-Draw Max

- Comprehensive diagramming tool
- Predefined DFD shapes and symbols
- Easy drag-and-drop functionality
- Export to multiple formats

10 Steps to Draw a DFD

1–5

How to Draw a DFD Online

Select a DFD template — choose "Blank ERD & Data Flow" from the template library

Name the data flow diagram — give it a clear, descriptive title

Add an external entity — place the entity that starts the process

Add a Process — drag a circle/oval onto the canvas to represent a process

Add a data store — place an open-ended rectangle to represent stored data

10 Steps to Draw a DFD

6–10

How to Draw a DFD Online

Continue adding items — add more processes, entities, and data stores as needed

Add data flows — drag arrows between symbols to show data movement direction

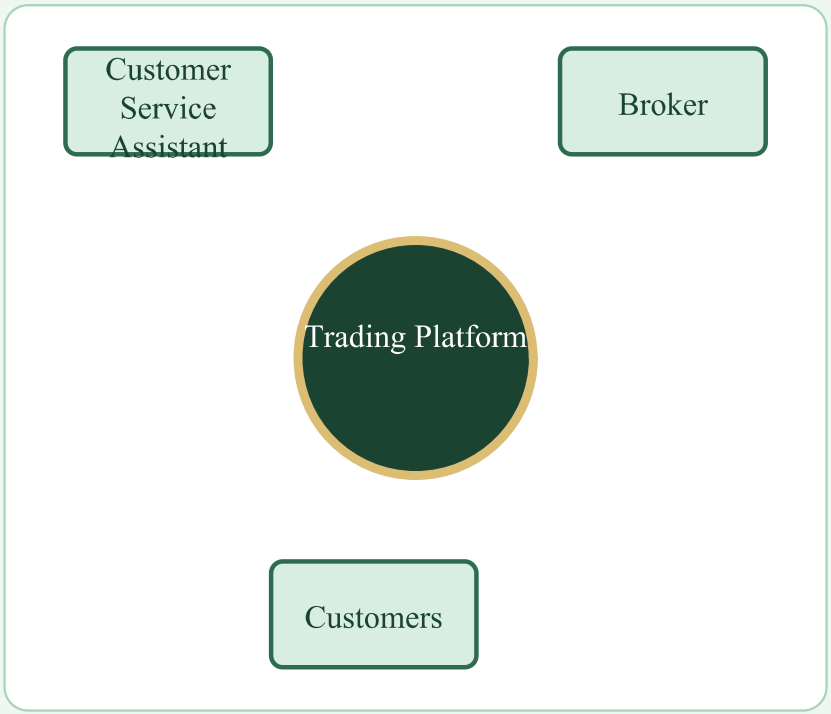
Name the data flows — describe what data is being moved along each arrow

Customize the diagram — apply colors, fonts, and formatting for clarity

Add a title and share — finalize with a clear title and export or share the DFD

DFD Example — Level 0: Securities Trading

LEVEL 0



Context Diagram Description

External Entities: Customer Service Assistant, Broker, Customers

Central Process: Single process — Trading Platform

Data flows in **both directions** between Customers and the Platform

Data flows in **one direction** from the Customer Service Assistant to the Platform

Data flows in **one direction** from the Broker to the Platform

DFD Example — Level 1: Customer Process

LEVEL 1

Decomposing Customer Interaction into Sub-Processes

Breaks down the customer process from Level 0 into **sub-processes**

- Shows data stores and data flows associated with each sub-process
- Reveals internal structure of how customer requests are handled

1. Account Creation	2. Cash Withdrawals	3. Securities Transactions
Register new customers	Process fund requests	Buy and sell orders

DFD Example — Level 2: Place Order

LEVEL 2

Decomposing the "Place Order" Sub-Process

- Decomposes the "Place Order" process from Level 1 into specific steps

Customer places order **OR** Broker places order

Orders are forwarded to the third-party **Stock Exchange Center**

- Transaction details forwarded after order is placed
- Shows detailed data flow within one specific process



Points to Remember: DFD

- A DFD is a **visual representation** of information flows within a system
- Elements: **Processes, Data Flows, Data Stores, External Entities**
- Each process should have at least **one input and one output**
- Each data store should have at least **one data flow in and out**
- Data stored in a system must go through a **process**
- All processes in a DFD go to **another process or a data store**

3.2

Application of Physical Data Model

Understanding database objects, constraints, and physical design implementation

What are Database Objects?

In a DBMS, various objects are used to **store**, **manage**, and **manipulate** data. Understanding these objects is essential for physical database design.

- **Tables** — Fundamental storage
- **Views** — Virtual tables
- **Indexes** — Speed optimization
- **Stored Procedures** — Precompiled SQL
- **Functions** — Return single values
- **Triggers** — Event handlers
- **Schemas** — Logical containers
- **Sequences** — Unique number generators
- **Synonyms** — Aliases for objects
- **Constraints** — Data integrity rules

Tables & Views

Tables

Fundamental objects storing data in **rows and columns**

- Columns define attributes (Name, Age, etc.)
- Each row = single record/entry
- The core building block of any database

```
CREATE TABLE Students (
 StudentID INT PRIMARY KEY,
 Name VARCHAR(100),
 Age INT
);
```

Views

Virtual tables representing result of a query

- Do NOT store data themselves
- Present data from one or more tables
- Simplify complex queries
- Enhance security by restricting access

```
CREATE VIEW ActiveStudents AS
SELECT Name, Age
FROM Students
WHERE Age > 18;
```

Indexes & Stored Procedures

Indexes

Improve **speed of data retrieval** operations

Unique Indexes — no duplicate values allowed

Composite Indexes — span multiple columns

- Trade-off: faster reads, slightly slower writes

```
-- Unique Index
CREATE UNIQUE INDEX idx_email
ON Students(Email);

-- Composite Index
CREATE INDEX idx_name_age
ON Students(LastName, Age);
```

Stored Procedures

Precompiled SQL statement collections

- Executed as a single unit
- Encapsulate business logic on the server
- Improve performance (reduce network traffic)

```
CREATE PROCEDURE GetStudent
 @ID INT
AS
BEGIN
 SELECT * FROM Students
 WHERE StudentID = @ID;
END;
```

Functions, Triggers & Schemas

F

Functions

- Similar to stored procedures

Return a **single value**

- Used in SQL expressions
- Perform calculations, manipulate data
- Can be scalar or table-valued

T

Triggers

- Special stored procedures
- Auto-execute on events (INSERT, UPDATE, DELETE)
- Enforce business rules
- Maintain audit trails
- Ensure data integrity automatically

S

Schemas

- Logical containers for DB objects
- Hold tables, views, procedures
- Organize related objects
- Enhance security and manageability
- Help with namespace management

Sequences, Synonyms & Constraints

Sequences

- Generate unique numeric values automatically
- Often used for primary key columns
- Auto-generate sequential numbers for new records

```
CREATE SEQUENCE seq_id
START WITH 1
INCREMENT BY 1;
```

Synonyms

- Aliases for database objects
- Simplify access using different names
- Useful for complex object names

```
CREATE SYNONYM emp
```

Constraints (4 Types)

Primary Key

Uniquely identifies each record in a table. Example: StudentID

Foreign Key

Ensures referential integrity between tables. Links related data.

Unique

All values in a column must be distinct. Example: Email address

Check

Enforces domain integrity by limiting allowed values. Example: Age >= 0

Constraints Detailed

Primary Key

Uniquely identifies each record in a table. No NULL values allowed. Only one per table.

```
StudentID INT PRIMARY KEY
```

Foreign Key

Ensures referential integrity between tables. References a primary key in another table.

```
BookID INT REFERENCES Books(BookID)
```

Unique Constraint

All values in the column must be distinct. Prevents duplicate entries in specific columns.

```
Email VARCHAR(100) UNIQUE
```

Check Constraint

Enforces domain integrity by limiting the values that can be placed in a column.

```
Age INT CHECK (Age >= 0)
```

Designing Database in E-Draw Max (Steps 1–4)

1 Open E-Draw Max

Launch the application on your computer

2 Create New Document

Go to File > New to start a new diagram project

3 Select ER Diagram Template

Search "ER Diagram" in the template library and choose a suitable template

4 Add Tables (Entities)

Create the following tables with their fields:

```
Authors: AuthorID (PK), Name, Birthdate
Books: BookID (PK), Title, AuthorID (FK), Genre, PublishedYear
Members: MemberID (PK), Name, Email, PhoneNumber
Loans: LoanID (PK), BookID (FK), MemberID (FK), LoanDate, ReturnDate
```

Designing Database in E-Draw Max (Steps 5–7)

5 Define Relationships

Establish the cardinality between tables:

Authors ↔ **1:M**Books
Members ↔ **1:M**Loans
Books ↔ **1:M**Loans

6 Arrange and Format

Adjust the layout to ensure tables are well-spaced and relationships are clearly visible. Use formatting options for a clean design.

7 Save Diagram

Go to File > Save As to save as E-Draw Max file, PDF, or image format.

Points to Remember: Physical Data Model

- Physical database design = **defining database objects and relationships**
- Main objects: **tables, indexes, sequences, saved searches, views**
- Tables contain **fields (columns)** and **rows (records)**
- Every table has an **entity/primary key** identifying each row
- **Foreign keys** create relationships between tables
- Relationships enable **powerful queries** across tables

3.3

Documentation of System Design

Creating comprehensive system and user documentation for your designs

What is System Design Documentation?

The System Design Document provides a comprehensive description of the system for developers, testers, and stakeholders.

- System requirements
- Operating environment
- System and subsystem architecture
- Files and database design
- Input formats, output layouts
- Human-machine interfaces
- Detailed design, processing logic
- External interfaces

Types of Documentation (5 Types)

01
System Design Document (SDD)

Describes functional and non-functional requirements, system architecture, and overall design approach.

02
Functional Specification Document (FSD)

Detailed information about system functionality, behavior, and what the system should accomplish.

03
Technical Specification Document (TSD)

Technical details including tools, technologies, programming languages, and implementation approach.

04
Database Design Document

Maps logical data model to the target DBMS with performance considerations and table structures.

05
Use Case Document

Standardized document describing use case steps, preconditions, expected outcomes, and interactions.

SDD & FSD Detailed

System Design Document (SDD)

Architecture & Requirements

Describes **functional and non-functional requirements**

- Records findings from the Requirements Document

Includes **High-Level Technical Design**

- Preliminary data design documented

Comprehensive **system architecture guide**

Functional Specification Document (FSD)

System Functionality & Behavior

Written by the project's **Business Analyst**

Detailed information on **system functionality**

- Based on requested behavior from stakeholders

Describes **how the system solution will function**

- Serves as a contract between business and development

TSD, Database Design & Use Case Document

T

Technical Specification (TSD)

- Technical attributes and requirements
- Tools, technologies, programming languages
- Intended user experience
- Product features and functionality
- Roadmap for engineers and developers

DB

Database Design Document

- Maps logical data model to target DBMS
- Considers system performance requirements
- Table structures and relationships defined
- Indexing strategies and optimization
- Data migration considerations

UC

Use Case Document

- Standardized format for use cases
- Steps, preconditions, and assumptions
- Expected outcomes and results
- Describes specific interaction scenarios
- Actor and system boundary definitions

System Documentation with GitHub Pages

STEPS 1–7

1. **Create a GitHub Repository**
   Name: **backend-system-docs**
2. **Enable GitHub Pages**
   Settings > Pages > select branch (main)
3. **Choose a Documentation Framework**
   Jekyll (built into GitHub Pages) or MkDocs / Docusaurus
4. **Set Up Local Environment**
   Install Ruby and Jekyll, create new Jekyll site
5. **Organize Documentation Structure**

/_config.yml	/index.md	/architecture.md	/api.md	/database.md	/deployment.md	/troubleshooting.md
6. **Write Documentation using Markdown**
   Cover: Overview, Architecture, API, Database Schema, Deployment, Troubleshooting
7. **Add Visuals**
   Include diagrams, flowcharts, UML in the /assets directory

System Documentation with GitHub Pages

STEPS 8–13

8 **Customize Your Site**
Modify **_config.yml** settings and choose a theme for your documentation site

9 **Preview Locally**
Run **bundle exec jekyll serve** and visit **localhost:4000** to preview

10 **Push Changes to GitHub**
Use **git add .**, **git commit -m "docs"**, **git push origin main**

11 **Access Documentation**
Visit **https://username.github.io/backend-system-docs**

12 **Maintain and Update**
Make regular updates and use version control for all changes

13 **Promote Documentation Usage**
Share the link with your team and encourage feedback

User Documentation

1–7

User Documentation with GitHub Pages

1. **Create GitHub Repository**
   Name: **user-documentation**
2. **Enable GitHub Pages**
   Settings > Pages > select branch
3. **Choose Framework**
   Jekyll (built-in to GitHub Pages)
4. **Set Up Local Environment**
   Install Ruby + Jekyll on your machine
5. **Organize Structure**

/_config.yml	/index.md	/getting-started.md	/features.md	/user-interface.md	/troubleshooting.md	/faqs.md

6. **Write Content**
   Introduction, Getting Started, Features, User Interface, Troubleshooting, FAQs
7. **Add Visuals**
   Screenshots and diagrams in /assets directory

User Documentation

8–14

User Documentation with GitHub Pages (continued)

8 **Customize Site**
Modify **_config.yml** settings and choose a theme

9 **Preview Locally**
Run **bundle exec jekyll serve** to preview

10 **Push to GitHub**
Use **git add**, **git commit**, **git push** to deploy

11 **Access Online**
Visit **https://username.github.io/user-documentation**

12 **Maintain and Update Regularly**
Keep documentation current with system changes

13 **Promote Documentation Usage**
Share the documentation link with all users

14 **Gather Feedback**
Use GitHub issues and feedback forms for user input

Points to Remember: Documentation

- **SDD:** Detailed system architecture, components, interfaces, and data flow
- **FSD:** System functionality, behavior, and what the system should do
- **TSD:** Technical details — hardware, software, and network requirements
- **Database Design Document:** Tables, relationships, and constraints
- **Use Case Document:** Interaction scenarios and user-system goals
- **User Documentation:** Instructions for end-users on system features
- **System Documentation on GitHub Pages** follows 13 structured steps
- **User Documentation** follows 14 structured steps

Key Takeaways

3.1

Data Flow Diagrams

Visual tool for data flow representation across **Levels 0, 1, and 2**

3.2

Physical Data Model

10 database objects from Tables to Constraints

3.3

System Documentation

5 document types: SDD, FSD, TSD, DB Doc, Use Case

GP

GitHub Pages

Platform for publishing both **system** and **user** documentation



Thank You

Questions & Answers

SWDBS401 Backend System Design — RQF Level 4